

1. Aim

To find the length of the **Longest Palindromic Subsequence (LPS)** of a given string using **Dynamic Programming**.

2. Problem Description

Given a string, find the longest subsequence that forms a palindrome. The characters do not need to be continuous, but their original order must be maintained.

For example:

BBABCBCAB

The longest palindromic subsequence is:

BABCBAB

Therefore, the length is **7**.

3. Algorithm

1. Read the input string.
2. Create a 2D DP table $dp[i][j]$.
3. Set $dp[i][i] = 1$ because a single character is always a palindrome.
4. Compare characters at positions i and j .
5. If they are equal:
 $dp[i][j] = 2 + dp[i+1][j-1]$
6. If they are different:
 $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$
7. The answer is stored in $dp[0][n-1]$.

4. Java Program

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String s = sc.nextLine();

        int n = s.length();
```

```

int[][] dp = new int[n][n];

for (int i = 0; i < n; i++)
    dp[i][i] = 1;

for (int len = 2; len <= n; len++) {
    for (int i = 0; i <= n - len; i++) {
        int j = i + len - 1;

        if (s.charAt(i) == s.charAt(j)) {
            dp[i][j] = 2 + (len == 2 ? 0 : dp[i + 1][j - 1]);
        } else {
            dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
}

System.out.println("Length = " + dp[0][n - 1]);
}
}

```

5. Sample Input

BBABCBCAB

6. Sample Output

Length = 7

7. Complexity

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n^2)$

8. Result

Thus, the program successfully finds the **Longest Palindromic Subsequence** using Dynamic Programming. For the input BBABCBCAB, the maximum palindrome subsequence has a length of **7**.